MongoDB-Based Application for Big Data Storage & Processing

Blood Bank Inventory Matching Network | 15 Marks

---

1. Problem Statement

India operates 3,000+ blood banks, largely in isolation with no live cross-bank visibility. This causes two recurring failures:

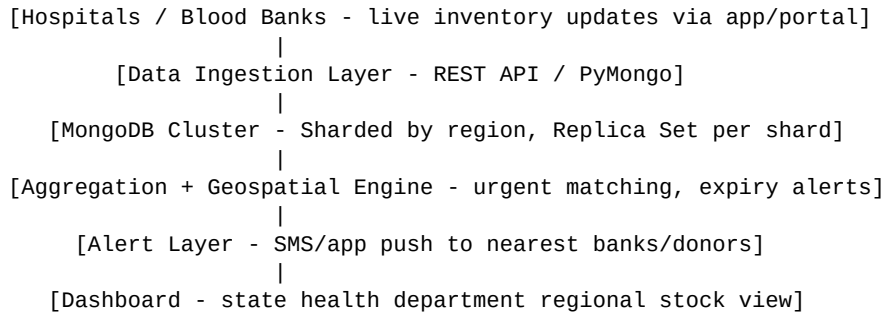
- **Shortage during emergencies** — nearby banks may have surplus stock of a needed blood type, but there is no real-time discovery mechanism, costing critical response time.
- **Expiry wastage** — blood units expire in 35–42 days; banks with soon-to-expire surplus have no way to redirect it to hospitals with active demand.

A MongoDB-based system solves this by ingesting live inventory data from all banks and using geospatial and aggregation queries to match supply to demand in real time.

2. Why MongoDB Fits

Requirement	MongoDB Feature
Live inventory updates from many banks	High write throughput, flexible document model
Varying data per bank (extra fields, formats)	Schema-less documents — no rigid table structure
“Nearest bank with this blood type”	Geospatial indexing (2dsphere, $geoNear)
Multi-bank, multi-region scale	Sharding across regions
Fault tolerance (banks must never lose data)	Replica sets
Trend analysis (expiry, demand spikes)	Aggregation Framework

3. System Architecture



4. Document Schema Design

```
{
  "bank_id": "BB2201",
  "name": "Victoria Hospital Blood Bank",
  "location": { "type": "Point", "coordinates": [77.5946, 12.9716] },
  "inventory": [
    { "blood_type": "O-", "units": 12, "nearest_expiry": "2026-09-05" },
    { "blood_type": "AB+", "units": 3, "nearest_expiry": "2026-09-10" }
  ],
```

```
  "cold_chain_status": "compliant",
  "last_updated": "2026-08-27T09:00:00Z"
}
```

- **Embedded array** for inventory (avoids joins, reads fast per bank)
- **GeoJSON Point** for location → enables 2dsphere indexing
- Index: `db.banks.createIndex({ location: "2dsphere" })`

5. Implementation

Step 1 – Setup & Ingestion (PyMongo)

```
from pymongo import MongoClient, GEOSPHERE

client = MongoClient("mongodb://localhost:27017/")
db = client["blood_bank_db"]
banks = db["banks"]

banks.create_index([("location", GEOSPHERE)])
banks.insert_many(bank_data_batch)  # bulk load from banks
```

Step 2 – Urgent Request Matching (core query)

Find nearest banks with a needed blood type, within 15 km, ranked by distance:

```
req_lng, req_lat = 77.6033, 12.9698   # requesting hospital's location

pipeline = [
  {"$geoNear": {
      "near": {"type": "Point", "coordinates": [req_lng, req_lat]},
      "distanceField": "distance",
      "maxDistance": 15000,
      "spherical": True
  }},
  {"$unwind": "$inventory"},
  {"$match": {"inventory.blood_type": "O-", "inventory.units": {"$gt": 0}}},
  {"$sort": {"distance": 1}},
  {"$limit": 5}
]
results = banks.aggregate(pipeline)
```

If no match is found within 15 km, the application logic re-runs with `maxDistance` expanded (radius escalation).

Step 3 – Expiry-Aware Redistribution

Flag units expiring within 5 days so they can be transferred to hospitals with active demand instead of being wasted:

```
from datetime import datetime, timedelta

soon = (datetime.utcnow() + timedelta(days=5)).isoformat()
pipeline = [
  {"$unwind": "$inventory"},
  {"$match": {"inventory.nearest_expiry": {"$lte": soon}}},
  {"$project": {"name": 1, "inventory.blood_type": 1,
                "inventory.units": 1, "inventory.nearest_expiry": 1}}
]
expiring_units = banks.aggregate(pipeline)
```

Step 4 – Scalability Setup

- **Sharding**: shard key = `region` (or geohash prefix) so banks in the same area sit on the same shard → fast local geospatial queries

- **Replica Set**: 1 primary + 2 secondaries per shard → no data loss if a bank's regional server goes down
- **Indexes**: 2dsphere on location, compound index on `{blood_type, nearest_expiry}` for fast expiry scans

Step 5 – Alerting

Aggregation results feed a notification service (Twilio/Firebase) that pushes alerts to nearby banks/donors when a request or expiry event triggers.

6. Extensions

- **Donor recall**: separate `donors` collection, match eligible donors (90-day gap rule) near a shortage zone
- **Demand forecasting**: time-series aggregation on historical requests to pre-position stock before accident-prone seasons/festivals
- **Rare blood registry**: national-level collection (Bombay blood group etc.) since rare types need state/national-radius search, not just local

7. Conclusion

The Blood Bank Inventory Matching Network demonstrates MongoDB's suitability for real-time, geographically-distributed big data applications. Its document model absorbs inventory data of varying structure across banks, geospatial indexing enables sub-second “nearest available stock” queries, and the aggregation framework supports both emergency matching and expiry-driven redistribution — directly reducing preventable shortages and wastage without needing a separate analytics engine.